PRD: Frontier Hypothesis Engine

Nov 7, 2026 · @Vidhi

Summary, users and success

We ship an agent workflow that takes one disease area (COPD) and returns 2 to 3 pursuits a biotech VC could act on this year, each answering the guidance's five questions with every claim cited and dated.

- **Users:** the UPMC Enterprises investment team. Secondary readers are the physicians and national experts in their 4 to 5 week diligence process, so the memo has a specialist and a generalist mode.
- **Problem:** good leads fall through because early diligence is slow and manual. The tool generates and screens hypotheses before a human team commits weeks to one.
- **Success, per the guidance:** actionable (a move this year), non-consensus (not crowded or priced in), evidence-backed (every claim cited and verified), honest about risk (bear case specific enough to direct diligence), traceable and calibrated (every claim traces to an agent; confidence matches evidence).
- **Success, per the mentor:** a strong competitor analysis; technology first (expansion after first approval, mechanistic rationale in other areas, the experiment that proves viability); a human in the loop; output that fits a Microsoft Office workflow.
- **Non-goals for Sunday:** UPMC-internal data (public sources only), devices and diagnostics at full depth, multi-user collaboration.

Coverage check

Of 22 requirements in the guidance, 5 are covered, 11 are partial, 5 were missing from our plan and 1 (the Copilot bonus) is a stretch. The biggest gaps: no genetics data for agent 2, no non-consensus measure, no catalysts, and the backtest marked as a stretch when it is a required submission.

Guidance requirement	Plan status	Fix
Five questions answered per hypothesis	Partial: agents produce the pieces, memo not organized by them	Memo section per hypothesis with the five questions as headings
One hypothesis at a time; 4 and 5 in parallel; one revise loop; citation gate	Covered	

Guidance requirement	Plan status	Fix
Every agent cites a source for each claim	Covered	
Agent 1: each shift's effect on trial time, cost or patient selection; negative shifts; precise segments	Partial	Fields for shift direction and effect; segments defined by inclusion criteria
Agent 2: evidence tiers (human genetics and precedent clinical data highest), converging human evidence, tractability, direction, safety clues	Gap: no genetics or tractability data	Add Open Targets (genetics, tractability, safety) to the store; map gold, silver, bronze onto the tiers
Agent 3: standard-of-care gaps expected at launch; minimum and holy-grail specs; 1 or 2 must-haves	Partial	Agent 3 reads approvals plus Phase 3 programs so the gap is judged at launch, not today
Agent 4: publications and patents; two ranked lists (repurposing, shelved) with stage and limitations	Gap: no drug or patent data	Add ChEMBL mechanisms and indications, terminated trials; patents for top mechanisms only
Agent 5: every disclosed program, orthogonal strategies, deprioritization flags	Partial	See Competitor analysis
Agent 6: non-reproduced findings, retractions, prior failures, safety and translation risk; kill criteria; cheapest test	Partial: no retraction data	Add Retraction Watch data (via Crossref) and terminated-trial reasons
Agent 7: real source, supports the claim, predates cutoff; claim ledger	Covered	
Agent 8: disagreements settled in the open; decision log; killed hypotheses	Partial: no decision log	Flow writes every revise, override and kill to a decisions table

Guidance requirement	Plan status	Fix
Pursuit: named asset, company or must-haves; gap to profile; main risk; what would change the call	Partial	Required fields in the memo schema
Upcoming catalysts	Gap	Competitor trial completion dates in the next 12 months, listed in the memo
"Proceed with Caution" flag	Gap	Score threshold; with one disease, compare against absolute bars and say so
Non-consensus ideas	Gap: no measure	Crowding score: programs on the mechanism plus publication volume
Calibrated confidence	Partial	Confidence on every claim, tied to its evidence tier
Backtest at January 1, 2021, beating the most-published baseline	Partial: marked stretch	Make it a must; score by hand against the COPD answer key
Build once, dated records, standard IDs, extract once, cost per agent and run	Covered	
Public sources only, dated	Covered	
Preference for Claude or GPT models	Partial: open models on Nebius	Agent 8 on OpenAI if free tokens are available; state the choice in the pitch
Submission: repo, live memo, backtest results, traces and cost log, pitch	Partial: backtest	Covered once the backtest is a must
Bonus: Copilot assessment	Stretch (mentor asked)	Comparison plan in the pitch; one agent 8 prompt run in Copilot if time allows

The five questions

Every surviving hypothesis in the memo answers all five, in this order, each answer built from the named agent's stored output.

Question	Answered by	Fields shown in the memo
1. Why is now the time to work on this disease?	Agent 1	Each shift since 2021 (or the 3 to 5 years before the cutoff): date, type (accepted endpoint, new patient segment, new trial tool, or a negative shift that raises the bar) and its effect on trial time, cost or patient selection; the patient segments, defined by inclusion criteria
2. Which mechanism(s) should be targeted?	Agent 2	Mechanism, direction (inhibit or activate), evidence tier and validation level (gold, silver, bronze), best-fit modality, tractability, safety clues, biggest uncertainty, mechanistic rationale in other conditions
3. What would a winning product look like?	Agent 3	Standard-of-care gap expected at launch; minimum-acceptable and holy-grail specs; the 1 or 2 must-haves; expansion opportunity after first approval
4. Which existing assets or companies could get there?	Agents 4 and 5	Repurposing list and shelved-asset list, each ranked by closeness to the product profile; active program table; crowding score; deprioritization flags
5. What could kill the hypothesis?	Agent 6, verified by agent 7	Verdict (survives, revise, kill); top risks with kill criteria; prior failures on the mechanism; the cheapest test of each risk, which is also the clinical experiment that would prove viability

Competitor analysis (mentor priority)

The mentor named competitor analysis as what matters most, so agent 5 is upgraded from a program list to a full landscape. Agent 4 feeds it the repurposing and shelved assets; agent 8 turns it into who to back or buy.

Two levels

- **Disease landscape, once per run:** approved standard of care, every disclosed program in the disease by mechanism and stage, and orthogonal strategies (a different mechanism for the same patients).
- **Per hypothesis:** programs on that mechanism, ranked by fit to agent 3's product profile.

Program record

Field	Source
Asset name and aliases (code names)	ChEMBL synonyms, ClinicalTrials.gov interventions
Owner and owner type (startup, mid-size, big pharma, academic)	ClinicalTrials.gov sponsor, SEC EDGAR
Mechanism and modality	ChEMBL, Open Targets
Stage and status, with the date they were true	ClinicalTrials.gov record history, Drugs@FDA
Lead trial: ID, size, primary endpoint, primary completion date	ClinicalTrials.gov
Novel edge, one sentence	Agent 5, from cited sources
Fit to the product profile (must-haves met: yes, partly, no)	Agent 5 against agent 3's output
Deprioritization flag and reason	Rules below
Funding signal for private companies	SEC Form D filings

Derived outputs

- **Landscape matrix:** mechanism by stage, with a count of programs in each cell.
- **Crowding score** per hypothesis, which feeds the non-consensus measure.
- **Deprioritization flags:** trial terminated or withdrawn for business reasons; no registry update within a window we set; asset dropped from the owner's latest pipeline disclosure; owner in a merger or strategic review. Flagged assets become access

candidates.

- **Catalysts:** competitor primary completion and regulatory dates in the next 12 months.
- **Investable list:** private companies working on the hypothesis, for invest pursuits.

Quality bars

- One asset, one row: aliases merged through standard drug IDs.
- A validator pass removes programs not truly in this disease or mechanism, the approach used in published due-diligence agents.
- Recall check: compare our count against a direct ClinicalTrials.gov count of interventional COPD trials by phase, and report the gap.
- In the backtest, registry versions as of the cutoff only.
- Every row cited; the full table exports to Excel for the Office workflow.

Acceptance for the live COPD run: the landscape includes the current standard of care, including the two 2024 approvals (ensifentrine and dupilumab), and every row carries a stage, a source and a date.

Agent requirements

Every agent returns schema-validated JSON, tags each claim with its agent, a confidence and a source ID, reads only through the as-of retrieval API, and logs tokens and cost per call.

Agent	Reads	Writes	Done when
1 Disease analysis	Disease, cutoff, intake settings; trials, approvals and literature from the 3 to 5 years before the as-of date	Why-now brief, shift records, patient segments	At least 3 shifts, each with its effect and a citation; negative shifts considered; segments defined by inclusion criteria
2 Mechanistic analysis	Segments; Open Targets genetics, tractability and safety; literature; trials	Ranked hypothesis cards	3 to 5 cards, each with a tier and validation level backed by cited evidence and an explicit direction

Agent	Reads	Writes	Done when
3 Optimal solution	Hypothesis cards; approvals and Phase 3 programs	One product profile per hypothesis	Must-haves are measurable (endpoint, effect size, dosing, safety)
4 Inactive-in-disease assets	Profile and mechanism; ChEMBL, Drugs@FDA, terminated trials, patents for top mechanisms	Repurposing list and shelved list	Each asset has a stage, last activity date, main limitation and profile fit
5 Active-in-disease assets	Profile and mechanism; ClinicalTrials.gov, ChEMBL, SEC EDGAR	Landscape, program table, crowding score, flags, catalysts	Meets the competitor analysis acceptance bar
6 Red Teamer	Outputs of agents 1 to 5; retractions; terminated-trial reasons	Verdict, top risks with kill criteria, cheapest test, pre-mortem	Every risk cited or labeled as reasoning; at most one revise round
7 Citation Checker	Every claim	Claim ledger: claim, agent, source ID, source date, supports (yes, partly, no), action taken	Every memo claim in the ledger; none dated after the cutoff
8 Orchestrator	All outputs, reviewer decisions	Ranking, pursuits, memo, decision log	2 to 3 pursuits; caution flag evaluated; every override logged

Recommendation memo

The guidance asks for a summary plus a section from every agent; the mentor asked for two pages. We do both: a two-page front a partner reads, and an appendix for diligence. Exported to Word, written for the audience chosen at intake (specialist or generalist).

Front, two pages

1. Executive summary: the 2 to 3 recommended pursuits in one line each, plus a "Proceed with Caution" flag and its reasons when the pursuits fall below our bars.
2. One block per pursuit:

    - Form: invest in a named private company, access a named asset (in-license, option, acquire), or build a new asset with its 1 or 2 must-haves.
    - The gap to the product profile, the main risk, and what would change the call.
    - Validation level (gold, silver, bronze) and crowding score.
3. Upcoming catalysts that could change the picture, with dates.

Appendix

1. The five questions answered for each surviving hypothesis, one section per agent layer.
2. Competitor landscape matrix and top program table.
3. Decision log: revise rounds, reviewer edits, disagreements and how each was settled.
4. Killed hypotheses with the reason and the kill criterion that fired.
5. Claim-level citation ledger and the run's cost summary.

Ask the memo (stretch)

A follow-up chat over a finished run, answering the mentor's request for back-and-forth instead of a single summary. It borrows the pattern of JP Morgan's Ask David: a supervisor agent routes each question to a sub-agent that reads the claim ledger, the competitor program table, or the evidence store at the run's as-of date, and every answer carries citations.

- **Kept separate from the main pipeline,** which stays deterministic so the agent order, the single revise loop and the citation gate are guaranteed.
- **Starting point:** langgraph-supervisor, an open-source LangGraph library for supervisor-style agents with memory and human-in-the-loop support.
- **Reference:** JP Morgan's talk on building Ask David with LangGraph. Ask David itself is internal, with no code released.
- **Owner:** whoever finishes their must items first on Sunday; it reads the stored run only, so it needs no changes to the main flow.

Data, backtest and efficiency

One COPD store from 2016 on serves both runs: the live run reads everything, the backtest reads only records dated before January 1, 2021.

Sources

Source	Feeds	Backtest note
PubMed abstracts	Agents 1, 2, 6	Filter by publication date

Source	Feeds	Backtest note
ClinicalTrials.gov with record history	Agents 1, 3, 4, 5, 6	Use each record's version as of the cutoff
Drugs@FDA and openFDA	Agents 3, 4, 5	Filter by approval date
Open Targets (genetics, tractability, safety)	Agent 2	Load a late-2020 release from the EBI archive
ChEMBL (mechanisms, indications, synonyms)	Agents 4, 5; ID matching	Use a release from before the cutoff
SEC EDGAR (10-K, 8-K, Form D)	Agent 5	Filter by filing date
Retraction Watch data (via Crossref)	Agent 6	Filter by retraction date
Patents (PatentSearch or Google Patents on BigQuery)	Agent 4, top mechanisms only	Filter by publication date

Efficiency

- Every extraction cached by document hash and prompt version, never redone.
- Standard IDs for genes, diseases, drugs, trials, patents and companies so records join across sources.
- Tokens and cost logged per agent per run; the pitch shows the backtest costing a fraction of the first run because it reuses the store.

Backtest procedure

1. Run with the as-of date set to January 1, 2021, single shot, no reviewer.
2. Record the recommended hypotheses and pursuits.
3. Build the naive baseline: the most-published COPD targets before the cutoff, counted from the store.
4. Score both against the answer key: a hit is a top hypothesis on the mechanism or patient segment of a later success; a good kill is a failure flagged with its real failure mode.
5. Run the contamination probe and report how much the models already know about post-2021 outcomes.

COPD answer key (confirm each in the store before scoring)

Outcome after the cutoff	Date	What a good January 2021 run would have said
Ensifentrine (Verona Pharma), a PDE3 and PDE4 inhibitor, approved for COPD maintenance	June 26, 2024	Access or invest: a novel inhaled mechanism in Phase 3
Merck agrees to acquire Verona for about $10 billion	July 9, 2025	Confirms the access pursuit was bankable
Dupilumab approved as the first biologic for COPD with an eosinophilic phenotype	September 27, 2024	Type 2 inflammation in a defined eosinophilic segment
Anti-alarmin programs report uneven Phase 3 data	2025 (to verify)	Survive with flagged replication risk, or rank lower

Sources: ensifentrine approval, dupilumab approval, Merck and Verona, anti-alarmin data.

Configurable settings

The meeting surfaced 3 explicit toggles plus 4 settings proposed during onboarding, so 7 to 8 controls in all. These were discussed as product ideas; none is confirmed as an MVP requirement yet.

#	Setting	Type	Purpose
1	Resource Constraints	Explicit toggle	Switch between resource-constrained and more flexible investment scenarios (e.g., limited budget vs. $10M available).
2	FDA-Approved / Repurposing Focus	Explicit toggle	Prioritize already FDA-approved drugs for repurposing versus considering unapproved or earlier-stage assets.
3	Output Audience / Technical Depth	Explicit toggle	Tailor the memo to clinicians, technical specialists, or general life sciences venture professionals.
4	Risk Tolerance	Proposed at onboarding	Adjust the preferred level of clinical validation and uncertainty.

#	Setting	Type	Purpose
5	Technology Focus	Proposed at onboarding	Select therapeutics, medical devices, diagnostics, etc.
6	Discovery Strategy	Proposed at onboarding	Prioritize existing technologies versus generating new therapeutic hypotheses.
7	Organizational Guardrails	Proposed at onboarding	Specify company-specific security restrictions, workflows, and permitted tools.
8	Output Length vs. Depth	Mentioned as possible	A possible additional preference raised in the meeting.

Team roles

Five owners, one folder each, so everyone codes and commits independently and the parts meet through the shared schemas.

Person	Owns	Folder	Done when
1: Data	Store schema (agreed with person 2 in the first hour); ingestion of PubMed, ClinicalTrials.gov, openFDA, Open Targets and ChEMBL; Nebius extraction and embeddings; naive baseline. Sunday: pitch deck, demo script and backtest scoring, with one slide from each person	/data, /store, /eval, /pitch	Dated COPD records queryable by Saturday night

Person	Owns	Folder	Done when
2: Platform and app	Agent schemas, fixtures and a stub retrieval API in the first hour; model router, cache, cost log, retrieval API with as-of date, run API (approve, edit and rerun at each checkpoint); agent 7 Citation Checker and claim ledger; Base44 app with the review screen; ArgosX scans	/core, /api, /agents/schemas.py, /agents/citations, /app	Schemas, fixtures and stub API pushed in the first hour
3: Analysis	Agent 1 disease analysis, agent 2 mechanisms, agent 3 product profile, agent 6 Red Teamer, validation-level rubric	/agents/analysis	Each agent passes on fixtures with the dev profile
4: Competitor	Agent 4 inactive assets; agent 5 full competitor landscape (program table, matrix, crowding score, deprioritization flags, catalysts)	/agents/competitor	Both agents pass on fixtures; the COPD landscape includes the 2024 approvals
5: Flow and memo	CrewAI Flow (order, parallel 4 and 5, one revise loop, guided and single-shot modes, citation gate), agent 8, Word and Excel export, decision log; integration owner who swaps in each real agent as it merges; runs the live and backtest runs	/flow, /memo	End-to-end run on stub agents by Saturday night

Working rules

- Each person works on their own branch (person-feature) and merges through a pull request one teammate reviews.

- CI runs schema and stub tests on every pull request, so main always runs end to end on stubs.
- Changes to agents/schemas.py go through person 2, since everyone codes against it.
- Person 5 runs the full flow after each merge, so integration happens continuously, not in one Sunday merge.

Checkpoints

1. Saturday, first hour: repo, schemas, fixtures and stub retrieval API (person 2); store schema agreed by persons 1 and 2.
2. Saturday night: COPD store loaded; each agent runs on fixtures; flow runs end to end on stubs.
3. Sunday 10am: first real end-to-end run on the dev profile; Base44 app live; first ArgosX Pro scan; pitch outline drafted.
4. Sunday noon: prompts frozen; final-profile live run and backtest run; memo exported; pitch filled with real outputs; demo recorded as a backup.
5. Sunday afternoon: mentor review; second ArgosX Pro scan after fixes, third just before judging.

Priorities for Sunday

Where this PRD differs from the Sunday plan in Tab 1, the PRD wins: the backtest run moves from stretch to must, and Open Targets and ChEMBL join the store.

Priority	Item	Owner
Must	Live COPD run in guided and single-shot modes; memo with the two-page front and appendix; Word export	Person 5
Must	Competitor landscape: program table, landscape matrix, crowding score, deprioritization flags, catalysts	Person 4
Must	Open Targets and ChEMBL added to the COPD store	Person 1
Must	Backtest run at January 1, 2021; naive baseline; hand scoring against the answer key	Person 5 runs it; person 1 builds the baseline and scores it
Must	Claim ledger, decision log, cost log per agent and run	Persons 2, 5

Priority	Item	Owner
Must	Pitch and demo script	Person 1, a slide from everyone
Should	SEC Form D funding signals, retraction data, patents for top mechanisms	Persons 1, 4
Should	Excel export of the program table; contamination probe; agent 8 on OpenAI	Persons 2, 5
Stretch	Copilot comparison, second disease, devices and diagnostics at full depth, Elo ranking, "Ask the memo" chat on LangGraph (see Recommendation memo), more cutoffs, Brier calibration score	Anyone free

Open decisions for the team

- [ ] Confirm COPD as the live disease (women's health is the backup).
- [ ] Set the bars that trigger "Proceed with Caution".
- [ ] Set the no-update window that flags a program as possibly deprioritized.
- [ ] Decide whether agent 8 runs on OpenAI, given the stated model preference.

Build specification

For building in Claude Code: save this tab as docs/PRD.md in the repo and point CLAUDE.md at it.

Stack and repo layout

Layer	Choice
Language and packaging	Python 3.11+, uv
API	FastAPI, Pydantic v2
Database	Postgres 16 with pgvector, run locally through Docker Compose
Orchestration	CrewAI Flows (plain asyncio fallback)
Model calls	LiteLLM behind our own router; Nebius Token Factory (OpenAI-compatible), OpenAI and Gemini as fallbacks

Layer	Choice
Exports	python-docx for the memo, openpyxl for the program table
Quality	pytest, ruff, GitHub Actions
App	Base44, calling the FastAPI service

```
frontier-engine/
  CLAUDE.md                  # points to docs/PRD.md, lists commands
  docs/PRD.md                # this tab
  pyproject.toml  .env.example  docker-compose.yml  Makefile
  core/                      # person 2: config, router, cache, costs, ids
    profiles.yaml            # dev and final model per agent
    prices.yaml              # list price per model, per 1M tokens
  store/                     # person 1: schema.sql, db.py
  data/                      # person 1: ingest_*.py, extract.py, embed.py,
baseline.py
  api/                       # person 2: main.py, retrieval.py, runs.py
  agents/
    schemas.py               # person 2: every agent input and output model
    prompts/                 # one .md per agent, fixed part first
    analysis/                # person 3: a1_disease, a2_mechanism,
a3_profile, a6_redteam
    competitor/              # person 4: a4_inactive, a5_active
    citations/               # person 2: a7_citations
  flow/                      # person 5: flow.py, state.py
  memo/                      # person 5: a8_orchestrator.py, render_docx.py,
export_xlsx.py
  eval/                      # person 1: backtest.py, answer_key.yaml,
probe.py
  tests/fixtures/            # one input and output JSON per agent
  pitch/                     # person 1
  app/                       # person 2: Base44 export
```

Data model

Every evidence row carries source_date; every run row carries run_id. Starting schema for store/schema.sql:

```
CREATE EXTENSION IF NOT EXISTS vector;

-- evidence (person 1)
CREATE TABLE documents (
```

```
  id text PRIMARY KEY,              -- 'pmid:123', 'nct:NCT0...', 'fda:...',
'sec:...', 'pat:...'
  source text NOT NULL,             -- pubmed | ctgov | fda | opentargets |
chembl | edgar | retraction | patent
  source_date date NOT NULL,        -- the date the as-of filter uses
  title text, body text, url text,
  text_hash text NOT NULL,
  embedding vector(4096)            -- Qwen3-Embedding-8B; no ANN index (see
note)
);
CREATE TABLE entities (
  id text PRIMARY KEY,              -- 'hgnc:...', 'mondo:...', 'chembl:...',
'nct:...', 'cik:...'
  kind text NOT NULL,               -- gene | disease | drug | trial | patent
| company
  name text NOT NULL, synonyms text[]
);
CREATE TABLE doc_entities (doc_id text REFERENCES documents, entity_id text
REFERENCES entities,
  PRIMARY KEY (doc_id, entity_id));
CREATE TABLE extractions (          -- extract once, never again
  text_hash text, prompt_version text, model text, output jsonb, created_at
timestamptz DEFAULT now(),
  PRIMARY KEY (text_hash, prompt_version));
CREATE TABLE programs (             -- one row per asset per disease, as of a
date (agent 5)
  id serial PRIMARY KEY, asset_entity text, owner_entity text, owner_type
text,
  mechanism text, modality text, stage text, status text, valid_from date,
valid_to date,
  lead_trial text, source_ids text[]);

-- runs (person 2 and person 5)
CREATE TABLE runs (id uuid PRIMARY KEY, config jsonb, status text, created_at
timestamptz DEFAULT now());
CREATE TABLE agent_outputs (run_id uuid, agent text, hypothesis_id text,
version int,
  output jsonb, status text,        -- draft | awaiting_review | approved |
superseded
  PRIMARY KEY (run_id, agent, hypothesis_id, version));
CREATE TABLE claims (id text PRIMARY KEY, run_id uuid, agent text,
hypothesis_id text,
  text text, kind text, confidence real, source_ids text[],
  ledger_status text);              -- unchecked | supported | partly |
unsupported | removed
CREATE TABLE decisions (id serial PRIMARY KEY, run_id uuid, at timestamptz
```

```
DEFAULT now(),
  actor text,                        -- reviewer | red_teamer | orchestrator
  action text, target text, reason text);  -- approve | edit | rerun | revise
| kill | override
CREATE TABLE llm_calls (id serial PRIMARY KEY, run_id uuid, agent text, model
text,
  input_tokens int, cached_tokens int, output_tokens int, list_price_usd
numeric,
  latency_ms int, cache_hit boolean, at timestamptz DEFAULT now());
```

Note: pgvector indexes cover at most 2,000 dimensions (4,000 for halfvec), so the 4,096-dimension embeddings stay unindexed. Exact search is fast enough for a COPD corpus of this size.

Interfaces

agents/schemas.py is the contract everyone codes against; person 2 pushes it in the first hour and changes go through a reviewed pull request. Starting models:

```
from datetime import date
from typing import Literal
from pydantic import BaseModel, Field

class RunConfig(BaseModel):
    disease: str                     # 'COPD'
    disease_id: str                  # MONDO or EFO ID
    as_of: date                      # today for live, 2021-01-01 for the
backtest
    mode: Literal['guided', 'single_shot']
    profile: Literal['dev', 'final']
    modality: Literal['therapeutic', 'device', 'diagnostic'] = 'therapeutic'
    risk_level: Literal['low', 'medium', 'high'] = 'medium'
    audience: Literal['specialist', 'generalist'] = 'generalist'
    constraints: str = ''            # resource limits typed at intake

class Claim(BaseModel):
    id: str
    agent: str
    text: str
    kind: Literal['evidence', 'reasoning']
    confidence: float = Field(ge=0, le=1)
    source_ids: list[str]            # document IDs; required when kind ==
'evidence'

class Shift(BaseModel):             # agent 1
```

```
    on: date
    type: Literal['endpoint', 'segment', 'trial_tool', 'negative']
    effect_on: list[Literal['time', 'cost', 'selection']]
    summary: str
    claims: list[Claim]

class Segment(BaseModel):
    name: str
    inclusion_criteria: list[str]
    claims: list[Claim]

class WhyNow(BaseModel):
    shifts: list[Shift]
    segments: list[Segment]

class HypothesisCard(BaseModel):     # agent 2
    id: str
    mechanism: str
    target_ids: list[str]
    direction: Literal['inhibit', 'activate', 'modulate']
    evidence_tier: Literal[1, 2, 3, 4]   # 1 = human genetics or precedent
clinical data
    validation_level: Literal['gold', 'silver', 'bronze']
    modality: str
    tractability: str
    safety_clues: list[str]
    biggest_uncertainty: str
    rationale_other_conditions: str
    claims: list[Claim]

class ProductProfile(BaseModel):     # agent 3
    hypothesis_id: str
    soc_gap_at_launch: str
    minimum_specs: list[str]
    holy_grail_specs: list[str]
    must_haves: list[str] = Field(min_length=1, max_length=2)
    expansion_opportunity: str
    claims: list[Claim]

class Asset(BaseModel):              # agent 4
    name: str
    entity_id: str
    owner: str
    stage: str
    last_activity: date | None
    limitations: str
```

```
    profile_fit: float = Field(ge=0, le=1)
    claims: list[Claim]

class InactiveAssets(BaseModel):
    hypothesis_id: str
    repurposing: list[Asset]
    shelved: list[Asset]

class Program(BaseModel):            # agent 5
    asset: str
    owner: str
    owner_type: Literal['startup', 'mid_size', 'big_pharma', 'academic']
    mechanism: str
    modality: str
    stage: str
    lead_trial: str | None
    novel_edge: str
    profile_fit: Literal['yes', 'partly', 'no']
    deprioritized: bool
    flag_reason: str | None
    funding_signal: str | None
    claims: list[Claim]

class Catalyst(BaseModel):
    on: date
    what: str
    source_ids: list[str]

class Landscape(BaseModel):
    hypothesis_id: str
    programs: list[Program]
    orthogonal: list[Program]
    crowding_score: float
    catalysts: list[Catalyst]

class Risk(BaseModel):               # agent 6
    text: str
    kill_criterion: str
    cheapest_test: str
    claims: list[Claim]

class Verdict(BaseModel):
    hypothesis_id: str
    verdict: Literal['survives', 'revise', 'kill']
    revise_target: str | None        # which agent to send back to, once
    risks: list[Risk]
```

```
    premortem: str

class LedgerEntry(BaseModel):        # agent 7
    claim_id: str
    source_id: str
    source_date: date | None
    supports: Literal['yes', 'partly', 'no', 'not_found', 'after_cutoff']
    action: Literal['keep', 'downgrade', 'remove']

class Pursuit(BaseModel):            # agent 8
    form: Literal['invest', 'access', 'build']
    name: str                        # company, asset, or the must-have
features
    hypothesis_id: str
    gap_to_profile: str
    main_risk: str
    what_would_change_the_call: str

class Memo(BaseModel):
    executive_summary: str
    pursuits: list[Pursuit] = Field(min_length=2, max_length=3)
    proceed_with_caution: bool
    caution_reasons: list[str]
    catalysts: list[Catalyst]
```

Agent function contract. Every agent is one async function, `async def run(inp: <Input>, ctx: RunContext) -> <Output>`. RunContext carries run_id, the RunConfig, a retrieval client already bound to as_of, and `ctx.llm(agent=..., messages=..., schema=...)`. The prompt lives in agents/prompts/<agent>.md with the fixed instructions first, so cached-input pricing applies. If the output fails schema validation, retry once with the error message, then fail loudly.

Services

Model router (core/router.py)

1. Look up the model for (agent, profile) in profiles.yaml.
2. Cache key = sha256 of model, messages, schema and temperature. On a hit, return the stored response and log cache_hit = true.
3. Call through LiteLLM with JSON-schema structured output. On a 429 or 5xx error, retry once, then walk the fallback list.
4. Log every call to llm_calls: tokens in, cached and out; list price from prices.yaml; latency.

5. Enforce caps from config: max output tokens per call, max calls per run, max USD per run. Breaching one raises BudgetExceeded and stops the run.
6. LLM_MODE=stub returns the fixture output for the agent, so tests and the flow run with no model calls.

```
# core/profiles.yaml (copy exact model ID strings from the Nebius console)
dev:
  default: nemotron-3.5-lightning
  a2_mechanism: deepseek-v4-flash-0731
  a3_profile: deepseek-v4-flash-0731
  a6_redteam: glm-5.3-flash
  a7_citations: glm-5.3-flash
  a8_orchestrator: deepseek-v4-flash-0731
final:
  extract: deepseek-v4-flash-0731
  embed: qwen3-embedding-8b
  a1_disease: deepseek-v4.1-flash-1017
  a4_inactive: deepseek-v4.1-flash-1017
  a5_active: deepseek-v4.1-flash-1017
  a2_mechanism: kimi-k3.1-pro
  a3_profile: kimi-k3.1-pro
  a8_orchestrator: kimi-k3.1-pro
  a6_redteam: glm-5.3
  a7_citations: glm-5.3-flash
  a7_escalate: kimi-k3.1-pro
fallbacks: [openai-mini, gemini-flash-lite]
```

Retrieval API (api/retrieval.py). Every endpoint takes as_of, and the server drops any record whose source_date is on or after it. Agents never call raw sources.

Endpoint	Returns
GET /search?q=&sources=&k=&as_of=	Hybrid keyword and vector search over documents, with source_date on every hit
GET /doc/{id}?as_of=	One document, or 404 if it postdates as_of
GET /trials?condition=&phase=&status=&as_of=	Trials as they stood on as_of
GET /targets/{disease_id}?as_of=	Open Targets associations, genetics, tractability and safety from the matching release
GET /drugs?mechanism=&indication=&as_of=	ChEMBL mechanisms, indications and maximum phase

Endpoint	Returns
GET /approvals?indication=&as_of=	Drugs@FDA approvals before as_of
GET /filings?company=&form=&as_of=	SEC filings (10-K, 8-K, Form D)
GET /retractions?doi=&as_of=	Retraction status

Run API for the app (api/runs.py)

Endpoint	Purpose
POST /runs	Start a run from a RunConfig; returns run_id
GET /runs/{id}	Status, current agent and hypothesis, cost so far
GET /runs/{id}/checkpoint	Output summary for the waiting agent, plus the automatic citation check
POST /runs/{id}/checkpoint	{action: approve, edit or rerun, edited_output?, comment?}; resumes the flow
GET /runs/{id}/memo.docx	Word memo
GET /runs/{id}/programs.xlsx	Competitor program table
GET /runs/{id}/trace	Agent outputs, decisions and the claim ledger
GET /runs/{id}/costs	llm_calls summed per agent

Flow and checkpoints

The flow is deterministic code; no model decides what runs next. Order, following the guidance:

```
start(RunConfig)
  a1_disease                          -> checkpoint
  a2_mechanism -> 3 to 5 cards        -> checkpoint
  for each card, one at a time:
    a3_profile                        -> checkpoint
    a4_inactive || a5_active          -> one checkpoint for both
    a6_redteam
      revise (first time only) -> rerun the agent named in revise_target,
then a6 again
      kill -> decisions + kill log, next card
                                      -> checkpoint
  a7_citations over every surviving claim   (gate: memo blocked until done)
  a8_orchestrator -> rank, pursuits, memo   -> final checkpoint (both modes)
  render memo.docx and programs.xlsx
```

Checkpoint mechanics

- State is a Pydantic RunState persisted with CrewAI's @persist. At a checkpoint in guided mode the flow saves state, marks the output awaiting_review and returns. POST /runs/{id}/checkpoint resumes it with kickoff(inputs={'id': run_id}).
- Single-shot mode treats every checkpoint except the final one as approved. The backtest forces single shot and skips the final review too.
- approve: mark the output approved and continue. edit: store the edited output as a new version, log the decision, continue. rerun: append the comment to that agent's prompt and run it again, at most twice.
- An approved change upstream marks every downstream output for that hypothesis superseded and reruns them.
- The review screen offers "approve the rest for this agent", so reviewing 3 to 5 cards does not take 3 to 5 separate stops each time.
- Every approve, edit, rerun, revise, kill and override writes a row to decisions; this table is the memo's decision log.

Ingestion

Each source gets one script in data/, idempotent and safe to rerun. Check each API's current docs before scripting; the access notes below are a starting point.

Source	Access	COPD query	source_date	Backtest handling
PubMed	NCBI E-utilities (esearch, efetch); an NCBI API key raises the rate limit	MeSH "Pulmonary Disease, Chronic Obstructive", 2016 to today	Publication date	Filter by date
ClinicalTrials.gov	API v2 studies endpoint	Condition COPD, all phases, interventional	First posted date	Status as of the cutoff: use an archived AACT snapshot from late 2020 if available; otherwise drop status changes and results posted after the cutoff
Drugs@FDA	openFDA drugsfda endpoint	Respiratory approvals, plus every drug that appears in the trials	Approval or submission date	Filter by date
Open Targets	Live: GraphQL API. Backtest: late-2020 release from the EBI FTP	COPD disease ID: associations, genetics, tractability, safety	Release date	Load the late-2020 release; older releases may use a different file format
ChEMBL	SQLite download per release	Mechanisms, indications, synonyms for every drug found above	Release date	Last release before 2021

Source	Access	COPD query	source_date	Backtest handling
SEC EDGAR	Full-text search and submissions JSON; SEC requires a descriptive User-Agent with a contact address	Companies found in trials and ChEMBL; 10-K, 8-K, Form D	Filing date	Filter by date
Retraction Watch	Free dataset distributed by Crossref	Every DOI in the store	Retraction date	Filter by date
Patents (should)	PatentSearch API (free key) or Google Patents public data on BigQuery	Top mechanisms only	Publication date	Filter by date

Pipeline per document

1. Fetch and store the raw record with source_date and text_hash.
2. Extract fields with a Nebius batch job; key on (text_hash, prompt_version) so nothing is extracted twice.
3. Normalize entities through ChEMBL synonyms, HGNC and MONDO/EFO IDs; write doc_entities.
4. Embed with Qwen3-Embedding-8B and store the vector.
5. Log ingestion counts per source; the recall check in the competitor section compares against them.

Config, testing and build order

```
# .env.example  (real values only in .env, never committed)
DATABASE_URL=postgresql://frontier:frontier@localhost:5432/frontier
PROFILE=dev                   # dev | final
LLM_MODE=live                 # live | stub
NEBIUS_API_KEY=
NEBIUS_BASE_URL=              # from the Nebius console
OPENAI_API_KEY=               # fallback
GEMINI_API_KEY=               # fallback
MAX_USD_PER_RUN=5
MAX_CALLS_PER_RUN=300
NCBI_API_KEY=
SEC_USER_AGENT="Team name contact@example.com"
```

```
# Makefile targets
make up          # docker compose up: Postgres with pgvector
make schema      # apply store/schema.sql
make ingest      # run data/ingest_*.py for COPD
make test        # ruff + pytest with LLM_MODE=stub
make run         # live COPD run, guided mode, dev profile
make backtest    # as_of 2021-01-01, single shot
make api         # FastAPI on :8000
```

Tests

- Schema tests: every fixture in tests/fixtures validates against agents/schemas.py.
- Agent tests: each agent runs on its fixture input with LLM_MODE=stub and a stub retrieval client.
- Leakage test: the retrieval API returns nothing dated on or after as_of, for every endpoint.
- Flow test: the full flow runs end to end on stubs in both modes, including one revise and one kill.
- Router test: cache hit on a repeated call, fallback on a simulated 429, BudgetExceeded at the cap.

CI: GitHub Actions runs make test on every pull request. A pull request merges only when CI passes and one teammate approves.

Conventions: prompts live in agents/prompts, never inline; no keys in code; every evidence claim carries source_ids; branches named person-feature; small pull requests.

Build order for Claude Code

1. Scaffold: repo layout, pyproject, docker-compose, Makefile, .env.example, CLAUDE.md. Done when make up and make test pass on an empty test suite.
2. Contracts: agents/schemas.py, one fixture pair per agent, stub retrieval client, LLM_MODE=stub. Done when schema tests pass.
3. Router, cache and cost log. Done when the router tests pass.
4. Store schema and ingestion for PubMed and ClinicalTrials.gov, then openFDA, Open Targets and ChEMBL. Done when dated COPD records load and make ingest is rerunnable.
5. Retrieval API with the as-of filter. Done when the leakage test passes.
6. Agents 1 to 7 against fixtures, then against the real store. Done when each meets its acceptance bar above.
7. Flow with both modes, checkpoints, revise and kill. Done when the flow test passes on stubs.
8. Agent 8, memo.docx and programs.xlsx. Done when a stub run produces both files.
9. Run API, then the Base44 app on top of it. Done when a guided run can be reviewed end to end in the app.
10. Final-profile live run, backtest run, baseline and scoring. Done when results are in eval/ and the pitch.